PHAT-JeT: Patch Hierarchical Attention Transformer for Efficient Particle Jet Tagging

Aaron Wang[1*] · Zihan Zhao[2*] · Alan Xia[2*] · Chang Sun[3] · Abhijith Gandrakota[4] · Jennifer Ngadiuba[4] · Richard Cavanaugh[1] · Javier Duarte[2]

[1]University of Illinois Chicago · [2]UC San Diego · [3]Caltech · [4]Fermi National Accelerator Laboratory · *Equal contribution

NEURAL INFORMATION PROCESSING SYSTEMS NeurIPS 2026

javierzhao.github.io/PHAT-JeT

1. Attention Under a Hard Budget

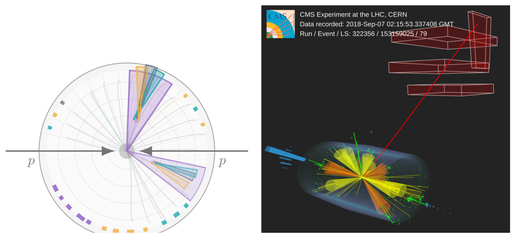
Classify an **unordered set of up to 150 points** with about **1M FLOPs** and **under 100 ns** on an FPGA.

- Full self-attention is $O(N^2)$: far too expensive.
- Efficient variants (low-rank, linear, hashing) fit the budget but drop exact pairwise interactions, and lose accuracy.

> *Can we keep exact attention, just not everywhere?*

Case study: the LHC trigger

40 million collisions per second; the Level-1 trigger decides within microseconds which events to keep. Its core task is **jet tagging**: naming the particle behind a spray of up to 150 detected particles.



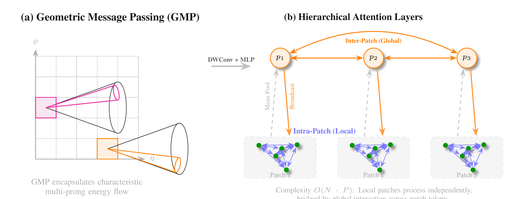
Left: jets (colored cones) leaving a proton–proton collision. Right: a real CMS event display.

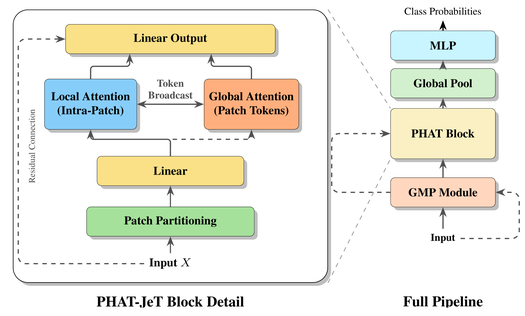
2. What Existing Methods Give Up

Method	Domain	Exact attention	Trigger-scale
Set Transformer	Sets	✗	✗
PaCa-ViT	Vision	✗	✗
Swin	Vision	✓	✗
ParT	Jets	✓	✗
JEDI-Linear	Jets	✗	✓
PHAT-JeT	**Jets**	✓	✓

- **Linformer:** low-rank projection compresses away particle–particle terms.
- **JEDI-Linear** (strongest FPGA baseline): linear-time, but no explicit pairwise edges.
- **HEPT:** hashing neighborhoods become unstable at small patch sizes.
- Swin and Longformer need a spatial grid or a meaningful sequence order; particle sets have neither.

3. PHAT-JeT in One Picture





GMP adds geometry, then one PHAT block runs exact attention inside patches and shares context through patch tokens.

4. Three Components

Geometric Message Passing `DWConv`

Bin particles onto a coarse (η, φ) grid, sum features per cell, apply a depthwise 2D conv, read back at each particle, and add residually. Encodes multi-prong structure cheaply.

Local patch attention `O(N·P)`

Split N particles into patches of size P; exact multi-head attention inside each. With $P = 10$: 15 patches, about 10% of full attention's computations.

Global patch tokens `O((N/P)²)`

Mean-pool each patch into a token, attend across tokens, broadcast back to every particle.

Patch attention alone already **beats full attention**: 81.51% vs. 81.27% at roughly half the FLOPs.

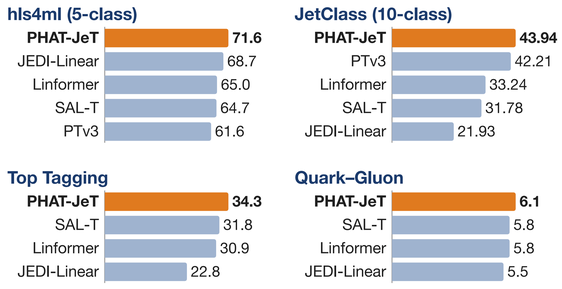
5. Results on JetClass

10-class, 128 particles per jet; efficient models at ~1.2–1.5M FLOPs, trained on 2M jets, tested on 20M.

Model	Acc (%)	AUC	Bkg Rej	Params	FLOPs
PHAT-JeT	**65.38 ± 0.08**	**0.9391**	**43.94 ± 0.65**	**6.8K**	**1.24M**
Point Transformer V3	64.69 ± 0.11	0.9372	42.21 ± 0.25	23.7K	1.45M
Linformer	62.64 ± 0.19	0.9305	33.24 ± 0.11	9.5K	1.49M
SAL-T	62.19 ± 0.28	0.9291	31.78 ± 0.90	7.0K	1.42M
JEDI-Linear	59.52 ± 0.82	0.9192	21.93 ± 1.64	6.1K	1.31M
Small ParT	57.58 ± 0.28	0.9115	15.83 ± 0.43	2.9K	1.52M
Transformer	56.18 ± 1.34	0.9056	15.09 ± 2.53	1.0K	1.24M
PointNet	55.87 ± 0.16	0.9063	20.83 ± 0.70	9.9K	1.35M
Unconstrained references (10^2–10^4× more FLOPs)					
ParT (full)	*83.6*	*0.9834*	*2345.6*	*2,140K*	*340M*
L-GATr	*83.9*	*0.9842*	*2844.2*	*1,682K*	*9,709M*

Best at this budget on the hardest benchmark: **+0.69 pts** accuracy over PTv3 (with 3.5× fewer parameters) and ≥2.7 pts over every other efficient model.

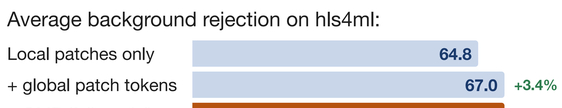
6. Best on All Four Benchmarks



Avg. background rejection (1 / false-positive rate at 80% signal efficiency; higher is better), all models at ~1–1.5M FLOPs. Each panel has its own scale; PTv3 is reported only on hls4ml and JetClass.

7. What Each Part Buys

Average background rejection on hls4ml:



Accuracy: 81.23 → 81.51 → 81.80%.

8. Beyond Jets: Real 3D Scans

ScanObjectNN PB_T50_RS (1024 points, coordinates only). Same PHAT-JeT block, with a 3D voxel grid for GMP.

Method	Params (M)	GFLOPs	OA (%)
PointNet	3.5	0.9	68.2
PointNet++	1.5	1.7	77.9
DGCNN	1.8	4.8	78.1
PointCNN	0.6	—	78.5
SimpleView	0.8	—	80.5 ± 0.3
PHAT-JeT-XS	**0.157**	**0.23**	**81.22 ± 0.16**
PHAT-JeT-S	**0.612**	**0.90**	**82.29 ± 0.41**
MVTN	3.5	1.8	82.8
PointMLP-elite	0.68	—	83.8 ± 0.6
PointMLP	13.2	31.3	85.4 ± 1.3
PointNeXt-S	1.4	1.6	87.7 ± 0.4

Beats PointNet++ and SimpleView with **0.157M parameters**. Baselines from published tables.

9. On an FPGA

AMD VU13P, 300 MHz, 64 particles; pre-route estimates.

Design	Acc	Bkg Rej	LUTs	ns
JEDI-Linear, official	80.94	66.0	69.9k	63.3
JEDI-Linear, official perm-invariant	81.81	80.4	158k	73.3
JEDI + GMP	82.08	84.8	144k	69.9
JEDI + GMP + patch tokens	81.93	84.0	208k	96.6
Patch attn (ReLU) + GMP	81.10	66.7	708k	99.9

Each component has a hardware variant (ReLU instead of softmax) that meets the **100 ns** target.

Official JEDI-Linear designs use 64 particles; its performance degrades when extended to 128.

10. Scope & Limitations

- Any **consistent ordering** works (k_T, p_T, Morton, random); detector inputs already arrive p_T-sorted. Train/test mismatch hurts.
- Jet benchmarks are simulated; the GMP grid must be tuned per detector.

TL;DR: exact attention inside small patches, linked by patch tokens, leads every trigger-scale model on four benchmarks.